C:\hpc-23-feb-2026\.\35\client.c

```
#include<stdio.h>
#include<sys/socket.h>
#include<string.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<unistd.h>
int main()
{
int file_descriptor;
struct sockaddr_in server_address;
char buffer[1025];
int n;
file_descriptor=socket(AF_INET,SOCK_STREAM,0);
if(file_descriptor<0)
{
printf("Unable to create socket\n");
return 1;
}
// initialize all the bytes of server_address structure to zero
bzero(&server_address,sizeof(server_address));
server_address.sin_family=AF_INET;
server_address.sin_port=htons(3500);  // server port
if(inet_pton(AF_INET,"127.0.0.1",&server_address.sin_addr)<0)
{
printf("Unable to setup server ip\n");
return 1;
}
if(connect(file_descriptor,(struct sockaddr *)&server_address,sizeof(server_address))<0)
{
printf("Unable to connect to server\n");
return 1;
}
n=read(file_descriptor,buffer,1024);
if(n==0)
{
printf("Server is not saying anything\n");
}
else
{
buffer[n]='\0';
printf("Server said [%s]\n",buffer);
}
return 0;
}
```

C:\hpc-23-feb-2026\.\35\server.c

```
#include<sys/socket.h>
#include<sys/epoll.h>
#include<arpa/inet.h>
#include<netinet/in.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<stdio.h>
int main()
{
int server_file_descriptor;
int client_file_descriptor;
int epoll_file_descriptor;
struct epoll_event ev; // to add a event listener, to specify what events to register
struct epoll_event events[20]; // to accept triggered events
int number_of_events;
int i;
char buffer[1025];
struct sockaddr_in server_sockaddr;
server_file_descriptor=socket(AF_INET,SOCK_STREAM,0);
if(server_file_descriptor<0)
{
printf("Unable to setup socket\n");
return 1;
}
bzero(&server_sockaddr,sizeof(server_sockaddr));
server_sockaddr.sin_port=htons(3500);
server_sockaddr.sin_family=AF_INET;
server_sockaddr.sin_addr.s_addr=htonl(INADDR_ANY);
if(bind(server_file_descriptor,(struct sockaddr *)&server_sockaddr,sizeof(server_sockaddr))<0)
{
printf("Unable to bind socket to port 3500\n");
return 1;
}
if(listen(server_file_descriptor,SOMAXCONN)<0)
{
printf("Unable to put socket in listening mode\n");
}
epoll_file_descriptor=epoll_create1(0);
// register/add event listener for socket related to server_file_descriptor
ev.events=EPOLLIN;
ev.data.fd=server_file_descriptor;
epoll_ctl(epoll_file_descriptor,EPOLL_CTL_ADD,server_file_descriptor,&ev);
while(1)
{
number_of_events=epoll_wait(epoll_file_descriptor,events,20,-1); // epoll_wait will wait for events
to trigger
for(i=0;i<number_of_events;++i)
```

```
{
if(events[i].data.fd==server_file_descriptor)
{
client_file_descriptor=accept(server_file_descriptor,(struct sockaddr *)NULL,NULL);
printf("Connection accepted\n");
strcpy(buffer,"Welcome to server");
write(client_file_descriptor,buffer,strlen(buffer));
close(client_file_descriptor);
}
}
}
return 0;
}
```

Thinking Machines Classroom Codes

C:\hpc-23-feb-2026\.\35\server2.c

```
#include<sys/socket.h>
#include<sys/epoll.h>
#include<arpa/inet.h>
#include<netinet/in.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<stdio.h>
int main()
{
int server_file_descriptor;
int client_file_descriptor;
int epoll_file_descriptor;
struct epoll_event ev; // to add a event listener, to specify what events to register
struct epoll_event events[20]; // to accept triggered events
int number_of_events;
int i,bytes_read;
char buffer[1025];
struct sockaddr_in server_sockaddr;
server_file_descriptor=socket(AF_INET,SOCK_STREAM,0);
if(server_file_descriptor<0)
{
printf("Unable to setup socket\n");
return 1;
}
bzero(&server_sockaddr,sizeof(server_sockaddr));
server_sockaddr.sin_port=htons(3500);
server_sockaddr.sin_family=AF_INET;
server_sockaddr.sin_addr.s_addr=htonl(INADDR_ANY);
if(bind(server_file_descriptor,(struct sockaddr *)&server_sockaddr,sizeof(server_sockaddr))<0)
{
printf("Unable to bind socket to port 3500\n");
return 1;
}
if(listen(server_file_descriptor,SOMAXCONN)<0)
{
printf("Unable to put socket in listening mode\n");
}
epoll_file_descriptor=epoll_create1(0);
// register/add event listener related to server file descriptor
ev.events=EPOLLIN;
ev.data.fd=server_file_descriptor;
epoll_ctl(epoll_file_descriptor,EPOLL_CTL_ADD,server_file_descriptor,&ev);
// register/add event listener related to stdin
ev.events=EPOLLIN;
ev.data.fd=STDIN_FILENO;
epoll_ctl(epoll_file_descriptor,EPOLL_CTL_ADD,STDIN_FILENO,&ev);
while(1)
```

```
{
number_of_events=epoll_wait(epoll_file_descriptor,events,20,-1); // epoll_wait will wait for events
to trigger
for(i=0;i<number_of_events;++i)
{
if(events[i].data.fd==server_file_descriptor)
{
client_file_descriptor=accept(server_file_descriptor,(struct sockaddr *)NULL,NULL);
printf("Connection accepted\n");
strcpy(buffer,"Welcome to server");
write(client_file_descriptor,buffer,strlen(buffer));
close(client_file_descriptor);
}else if(events[i].data.fd==STDIN_FILENO)
{
// do not use any user space related function over here like scanf/getchar .....
bytes_read=read(STDIN_FILENO,buffer,sizeof(buffer)-1);
buffer[bytes_read]='\0';
printf("(%s) arrived from keyboard\n",buffer);
}
}
}
return 0;
}
```

C:\hpc-23-feb-2026\.\35\cpp_server.cpp

```
int main()
{
try
{
char buffer[1024];
ipv4_tcp_server server(3035);
while(1)
{
cout<<"Waiting for connection request....."<<endl;
client=server.accept();
bytes_read=client.receive(buffer,sizeof(buffer));
buffer[bytes_read]='\0';
client.send("sdfsdfsssfsfsfsfsfd");
client.close();
}
}catch(network_exception &ne)
{
cout<<ne.what()<<endl;
}
return 0;
}
```

Thinking Machines Classroom Codes